IUT Colmar
Institut Universitaire de Technologie
UNIVERSITÉ HAUTE-ALSACE

BUT Réseaux et Télécommunications

KIENTZ Alexandre

SITOVSKYI Illia

ProxmoxVE

Cahier des charges

SAE 3.02

Année universitaire :

2026 – 2027

Table des matières

1) Présentation du projet *3*

1. Contexte 3

2. Problématique 3

3. Solution proposée 3

2) Spécifications techniques *4*

1. Langage de programmation 4

2. Architecture du projet 4

3. Frameworks et bibliothèques 4

3. Environnement de développement 5

4. Base de données 5

5. Gestion du versioning 5

6. Compatibilité 5

3) Périmètre fonctionnel & « Coût humain » *6*

4) Planning et livrables *7*

1. Rendu des comptes rendus 7

2. Cahier des charges 7

3. Rendu final du projet 7

4. Planning Gantt 7

1) Présentation du projet

1. Contexte

Le client souhaite améliorer la circulation des véhicules de secours, notamment des ambulances, des véhicules de police et des sapeurs-pompiers, afin de leur permettre d'atteindre plus rapidement leur lieu d'intervention. Lors des périodes de forte affluence, la circulation peut être fortement ralentie et constituer un obstacle à la rapidité d'intervention des services d'urgence.

2. Problématique

Lorsqu'un véhicule d'urgence doit intervenir, les embouteillages et les carrefours très fréquentés peuvent considérablement augmenter son temps de trajet. Le besoin du client est donc de fluidifier et sécuriser le passage des véhicules d'urgence, en leur permettant de bénéficier temporairement d'une priorité aux intersections, tout en limitant les perturbations pour les autres usagers de la route.

3. Solution proposée

Nous proposons de développer un système de gestion intelligente des feux de circulation permettant de détecter l'arrivée d'un véhicule d'urgence et d'adapter temporairement le fonctionnement des feux sur son trajet.

Pour la maquette, le système sera représenté à l'échelle d'un carrefour équipé de feux tricolores. Lorsqu'un véhicule d'urgence est détecté, le système pourra modifier la signalisation afin de lui accorder la priorité et de faciliter son passage. Une fois le véhicule passé, les feux reviendront à leur fonctionnement normal.

L'objectif est ainsi de réduire le temps de trajet des services d'urgence tout en garantissant la sécurité des autres usagers.

2) Spécifications techniques

1. Langage de programmation

Le projet sera développé en **Python 3.14**, qui sera utilisé pour l'ensemble des applications composant le système.

2. Architecture du projet

L'architecture du projet reposera sur une communication client-serveur. Elle sera composée de la manière suivante :

- Un serveur, chargé de gérer les connexions, de centraliser les informations et de transmettre les commandes aux clients
- Un client représentant le carrefour, chargé de gérer et de simuler le fonctionnement d'un feu de circulation.
- Un client représentant un véhicule d'urgence, permettant de signaler une situation d'urgence et de demander une priorité au niveau de carrefour.
- Un client représentant les véhicules des usagers, permettant de simuler le traffic routier. Cette partie sera effectuée si le temps le permet, la priorité est affectée aux trois premières parties.

3. Frameworks et bibliothèques

Les principales librairies utilisées sont :

- Thread, pour la gestion du multithreading.
- Socket, pour la gestion des paquets réseaux.
- PyQT6, pour la gestion des interfaces graphiques.

Cette liste pourra évoluer au cours du développement du projet en fonction des besoins.

3. Environnement de développement

Le développement du projet sera réalisé sur PyCharm ou Visual Studio Code.

Un environnement virtuel « venv » sera utilisé pour être sûrs d'utiliser bien les dépendances du projet et de faciliter son installation sur différentes machines.

Les dépendances seront regroupées dans un fichier « requirements.txt », pour également faciliter l'installation.

4. Base de données

Pour ce projet, nous utiliserons une base de données MySQL / MariaDB.

La base de données aura pour but de stocker des informations contenant les déclenchements d'urgences des feux.

5. Gestion du versioning

Pour le versioning le service retenu est Git et hébergé sur Github

https://github.com/AlexKientz67/SAE-3.02

6. Compatibilité

Le projet sera conçu pour être compatible avec les systèmes suivants :

- Windows
- Linux
- MacOS

3) Périmètre fonctionnel & « Coût humain »

Fonctionnalité	Priorité (MoSCoW)	Complexité (1 à 5)	Coût humain estimé
Mise en place de la communication Client < - > Serveur	Must Have	3	4 heures
Connexion et gestion du carrefour	Must Have	3	2 heures
Connexion et gestion des véhicules d'urgence	Must Have	2	4 heures
Gestion des feux	Must Have	1	2 heures
Interface graphique du serveur	Must Have	3	4 heures
Interface graphique des clients	Must Have	3	4 heures
Journalisation des évènements	Should Have	2	4 heures
Simulation des véhicules usagers	Could Have	5	5 heures
Documentation Technique	Must Have	2	2 heures

Charge totale estimé :

- 31 heures de travail par personne

- 62 heures de travail au total

4) Planning et livrables

1. Rendu des comptes rendus

- Dimanche 13 septembre, 1er compte rendu

- Lundi 21 septembre, 2e compte rendu

- Dimanche 4 octobre, 3e compte rendu

- Vendredi 30 octobre, 4e compte rendu

- Dimanche 15 novembre, 5e compte rendu

2. Cahier des charges

- Jeudi 24 septembre

3. Rendu final du projet

- Mercredi 15 novembre 2026

4. Planning Gantt

